Unicorn Engineering Interview Exercise

Product Matching Pipeline (with AI in the loop)

Candidate Instructions

Overview

You're building a system that receives product submissions from an external partner and resolves each one to a canonical product in our catalog. Submissions are messy: aliases, abbreviations, typos, missing fields, duplicates. Some matches are unambiguous and some require LLM judgment. The goal is to ingest, validate, deduplicate, and match these submissions safely.

You will use AI in two distinct ways:

1. As an authoring tool - Cursor, Claude Code, Copilot, etc. for writing code.
2. As a system component - an LLM call inside your pipeline, used where deterministic logic isn't enough.

We're evaluating how you structure the problem and how you reason about shipping a system that includes a non-deterministic, paid, sometimes-wrong dependency. We are not evaluating typing speed.

Provided Data

- **product_matching_submissions.json** - 32 incoming submissions (mix of clean, ambiguous, malformed, duplicate, near-duplicate)
- **inventory_existing_catalog.json** - 25 canonical catalog products across six categories
- **inventory_validity_rules.json** - required fields and field-level validation rules
- **product_matching_rules.json** - decision classes, attribute policies (size, vintage, category), duplicate policy, false-positive policy, cost assumption. Read this carefully — it locks down rules you'd otherwise have to guess.
- **product_matching_eval.json** - 32 hand-labeled records with expected_product_id and acceptable_decisions for measuring matcher quality

The exercise is tiered

We expect most candidates to finish Tier 1 cleanly, ship a partial Tier 2, and discuss Tier 3 in the interview. Tier 1 done well beats all tiers done sloppily. Stop when you hit ~2 hours and write up what you'd do next.

Tier 1 - Deterministic Core (target: ~45 min)

Build the pipeline that doesn't need an LLM:

- Ingest records from the file (assume the file can be larger than memory in production).
- Validate against the rules. Separate validation from persistence.
- Identify: valid, invalid, exact duplicates, near-duplicates, unmatched.
- Match valid records to the catalog via deterministic logic (exact keys, normalized strings, etc.).
- Produce a summary report.

Tier 1 is expected to leave the ambiguous submissions - aliases, abbreviations, producer mismatches - as unmatched. Tier 2 picks them up.

Constraints:

- Must not corrupt the existing catalog.
- Import may fail midway - partial progress should be safe.
- Treat the catalog as production: writes go through a staging boundary you define.

Tier 2 - LLM-Assisted Matching (target: ~45 min)

The deterministic matcher will leave a residual set of ambiguous records — examples in the actual data: "Ch. Margaux 2005" vs Chateau Margaux, "Pappy Van Winkle Fifteen" with producer "Van Winkle" (catalog says Old Rip Van Winkle), "Dom Romanee-Conti La Tache 2015" with producer "DRC".

For these, call an LLM to decide. Required:

- Each AI-assisted match emits a confidence score and a reason in structured output (use JSON mode / tool use / a schema - not free text parsing).
- A threshold policy: high-confidence → auto-apply, low-confidence → human-review queue (a JSON file is fine). You decide the threshold and justify it.
- Every LLM call is logged with input, output, model, and timestamp, sufficient for audit and replay.
- A deterministic fallback: if the LLM API is unavailable, the import still completes (records go to review queue, not to the floor).
- Idempotency: running the same input twice should not produce different write decisions. Think about how you achieve this when the underlying model is non-deterministic.

You do not need a UI for the review queue. A file is fine.

Tier 3 - Scale, Cost, and Evaluation (target: ~30 min, mostly written)

You will not implement this tier. In your README, answer:

1. Cost. You have a $50 LLM budget per import. The file is 1M records. Walk through where calls happen and how you'd keep total cost under budget. Where do you route to cheaper signals (embeddings, n-gram similarity, rules) before reaching the LLM?
2. Evaluation. Use product_matching_eval.json to report your matcher's accuracy as it stands, both product-id correctness and whether your emitted decision was in acceptable_decisions. The eval's scoring guidance weights false positives more heavily; reflect that in your reported metric. How would you catch regressions if someone changed the prompt next week?
3. Drift. How would you know in production if the LLM started making worse decisions over time?
4. Reversibility. A bad import made it to staging. What's your rollback story?

Deliverables

Code (GitHub repo preferred) plus a README containing:

1. Approach & tradeoffs - what you built, what you skipped, why.
2. Data flow diagram - one image or ASCII sketch.
3. Scaling plan - what breaks at 1M records and what you'd change.
4. Failure handling - partial failure, LLM downtime, bad data, replay.
5. AI usage - authoring. One prompt you iterated on. Show the bad first version, what was wrong with the output, and your revised version. One sentence on why.
6. AI usage - system component. Where in the pipeline does the LLM run, what schema does it return, what's the fallback, what does it cost per 1k records (estimate fine).
7. One thing the LLM got wrong during your development - how you noticed, what you changed.
8. Eval results on product_matching_eval.json: report (a) % of submissions where you returned the correct expected_product_id (treating null as a valid answer for no_match), and (b) % where your emitted decision was in acceptable_decisions. One-line interpretation.

Time Expectation

~2 hours of focused work. Tier 1 fully, Tier 2 at least one working path end-to-end, Tier 3 written. We'd rather see Tier 1 + half of Tier 2 done well than all tiers half-baked.